

Frage: Muss man `Thread.sleep()` im `CompletableFuture` fangen?

1. Kernfrage: Muss man `Thread.sleep()` im `CompletableFuture` fangen?

Ja. Auch wenn `CompletableFuture` oft damit beworben wird, dass man weniger `try-catch` Blöcke benötigt, gilt dies nicht für `Checked Exceptions` (wie `InterruptedException` von `Thread.sleep()`) innerhalb des Lambdas.

Der technische Hintergrund liegt in den funktionalen Interfaces:

Feature	Interface	Methode im Interface	Umgang mit Checked Exceptions
<code>Future<V></code>	<code>Callable<V></code>	<code>V call() throws Exception;</code>	Erlaubt. Exceptions dürfen geworfen werden (sie landen in einer <code>ExecutionException</code> beim Aufruf von <code>.get()</code>).
<code>CompletableFuture<T></code>	<code>Supplier<T></code>	<code>T get();</code>	Verboten. Die Methodensignatur erlaubt kein <code>throws</code> . <code>Checked Exceptions</code> <i>müssen</i> im Lambda gefangen werden.

Umgang mit Sleep im Chaining

Da ein `try-catch` Block mitten in einer Lambda-Expression den Lesefluss (besonders beim Chaining) stört, empfiehlt es sich, eine Hilfsmethode zu nutzen ("Unchecked Wrapper").

Die Hilfsmethode:

```
public static void sleepUnchecked(long millis) {
    try {
        Thread.sleep(millis);
    } catch (InterruptedException e) {
        // Umwandlung in RuntimeException, damit Supplier sie akzeptiert
        throw new RuntimeException(e);
    }
}
```

Anwendung im Chaining:

```
CompletableFuture.supplyAsync(() -> {
    // Kein try-catch Block nötig, da in sleepUnchecked versteckt
    sleepUnchecked(500);
    return 42;
})
.thenApply(ergebnis -> {
    // Auch hier bleibt der Code lesbar
    sleepUnchecked(200);
    return ergebnis * 2;
});
```

Vergleich der Fehlerbehandlung

Dies ist der eigentliche Vorteil von `CompletableFuture`: Man muss den Haupt-Thread nicht mit einem `try-catch` Block belasten, um auf Fehler der Berechnung zu reagieren.

Der klassische Weg (`Future<V>`)

Hier müssen wir beim Abholen des Ergebnisses (`.get()`) einen `try-catch` Block schreiben. Der Thread blockiert und wartet, ob ein Fehler fliegt.

```
Future<String> future = executor.submit(() -> {
    throw new RuntimeException("z.B. Datenbank nicht erreichbar!");
});

try {
    // Hier wird gewartet und POTENZIELL gefangen
    String result = future.get();
} catch (ExecutionException e) {
    // Fehlerbehandlung passiert hier im Haupt-Thread
    System.err.println("Fehler in der Berechnung");
} catch (InterruptedException e) {
    // Fehlerbehandlung für den Thread-Abbruch
}
```

Der moderne Weg (CompletableFuture<T>)

Hier definieren wir einen "Plan B" (.exceptionally), der automatisch ausgeführt wird, falls ein Fehler auftritt. Es ist kein try-catch um den Aufruf herum nötig.

```
CompletableFuture.supplyAsync(() -> {
    if (true) throw new RuntimeException("z.B. Datenbank nicht erreichbar!");
    return "Erfolg";
})
.exceptionally(ex -> {
    // Dieser Block wird NUR bei einem Fehler ausgeführt.
    // Wir können hier einen Default-Wert zurückgeben ("Recovery").
    System.err.println("Fehler aufgetreten: " + ex.getMessage());
    return "Standard-Wert";
})
.thenAccept(result -> {
    // Wird ausgeführt mit "Erfolg" ODER "Standard-Wert"
    System.out.println("Endergebnis: " + result);
});
```

Fazit:

Die Aussage in der Vorlesung bezieht sich auf die Behandlung des Ergebnisses, nicht auf den Inhalt des Lambdas.

- Bei Future<V>: Man muss beim Aufruf von .get() zwingend einen try-catch Block schreiben (für InterruptedException und ExecutionException).
- Bei CompletableFuture<T>: Man kann Fehler asynchron weiterleiten, ohne den Hauptthread mit try-catch zu blockieren (z. B. via .exceptionally() oder .handle()).
- **Weiterer Hinweis:** .get() vs. .join()

.get() (Legacy / Kompatibilität)

Da CompletableFuture das alte Future-Interface implementiert, muss es .get() anbieten.

- Exception: Wirft Checked Exceptions (ExecutionException, InterruptedException).
- Nachteil: Erfordert zwingend try-catch.
- Einsatz: Eher selten bei CompletableFutures, außer man braucht explizit Kompatibilität zu alten APIs.

.join() (Modern / CompletableFuture-Style)

- Dies ist die bevorzugte Methode für CompletableFuture.
- Exception: Wirft Unchecked Exceptions (CompletionException).

- Vorteil: Kein try-catch Zwang. Das macht `.join()` ideal für die Verwendung in Lambdas oder Java Streams, wo Checked Exceptions verboten oder unhandlich sind.

```
CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() ->
42);

//VARIANTE 1: .get()
try {
    Integer result = future.get();
} catch (InterruptedException | ExecutionException e) {
    // Muss behandelt werden
}

//VARIANTE 2: .join()
Integer result = future.join(); // Kein Try-Catch nötig!

// Beispiel in einem Stream (nur mit join möglich):
List<CompletableFuture<String>> futures = ...;
List<String> results = futures.stream()
    .map(CompletableFuture::join) // .get() würde hier einen Compiler-
    Fehler erzeugen!
    .collect(Collectors.toList());
```